Taller Comandos de Git

Iván Santiago Valencia Hernández

Ingeniería de Procesos de Software
Brayan Stiven Torres Ovalle

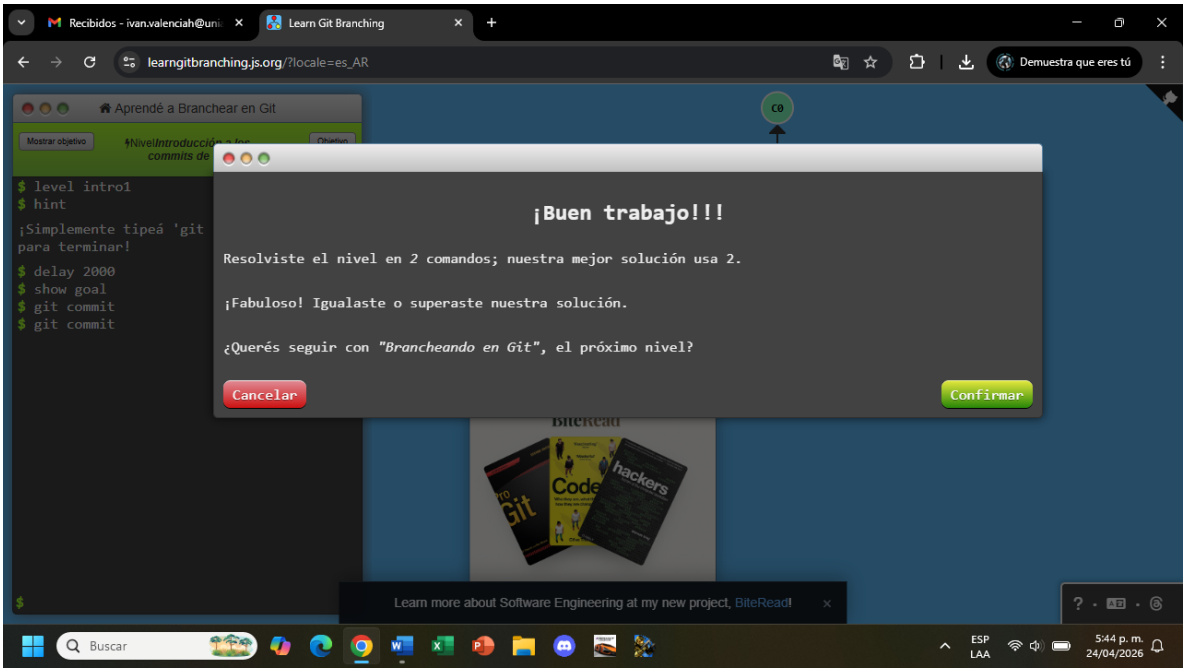
Ingeniería de Sistemas y Computación

Universitaria Agustiniana
Bogotá D.C
2026

Juego de Comandos Git

Pagina web de aprendizaje llamada Learn Git Branching donde se realizo el laboratorio: https://learngitbranching.js.org/?locale=es_AR

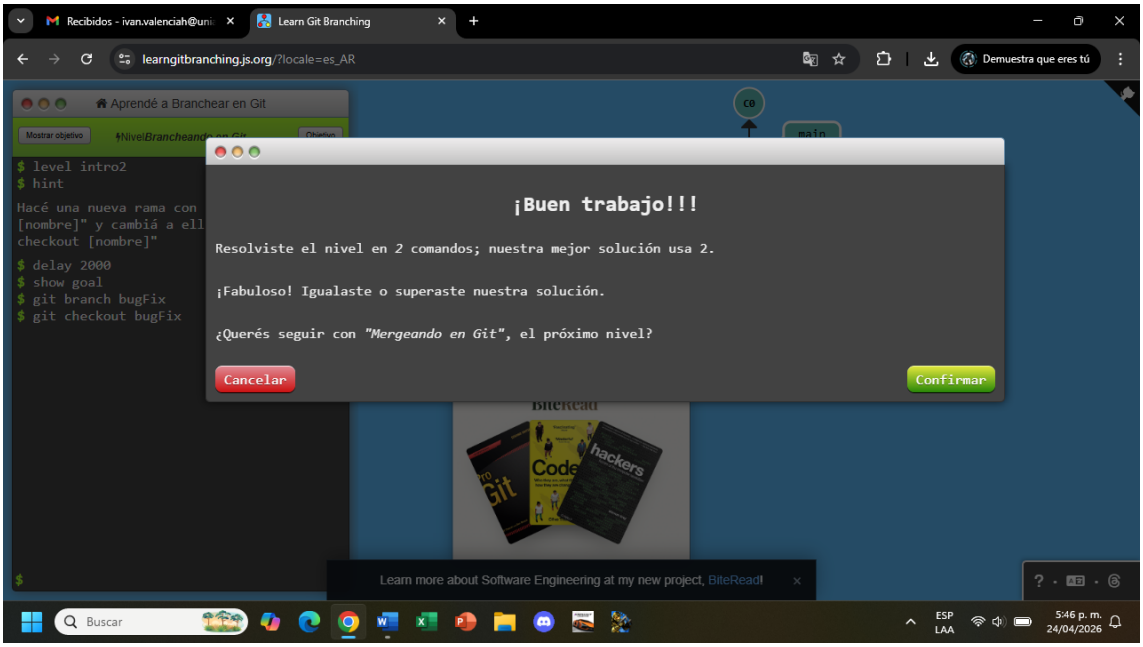
Empezamos con el primer capitulo llamada “Secuencia Introductoria” y el primer nivel es “Introducción a los comandos de Git”.



Despues de una breve explicación hay que desarrollar un desafío donde se plantea utilizar el comando git commit.

Un commit en un repositorio git registra un snapshot de todos los archivos en tu directorio. Es como un gran copy&paste, pero incluso mejor. Git mantiene también, un historial de qué commits se hicieron cuándo. Es por eso que la mayoría de los commits tienen commits ancestros arriba suyo -- designamos esto con flechas en nuestra visualización.

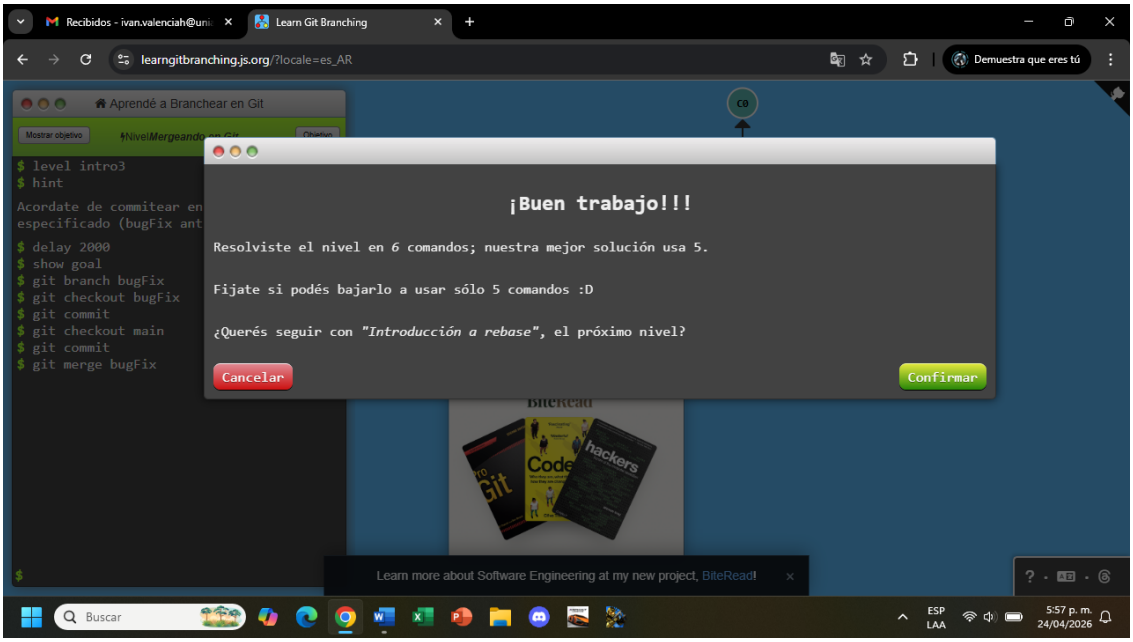
El segundo nivel es “Brancheando en Git”. Después de la explicación y ejemplos se realiza el desafío.



git branch “nombre”: Crea o muestra ramas de trabajo.

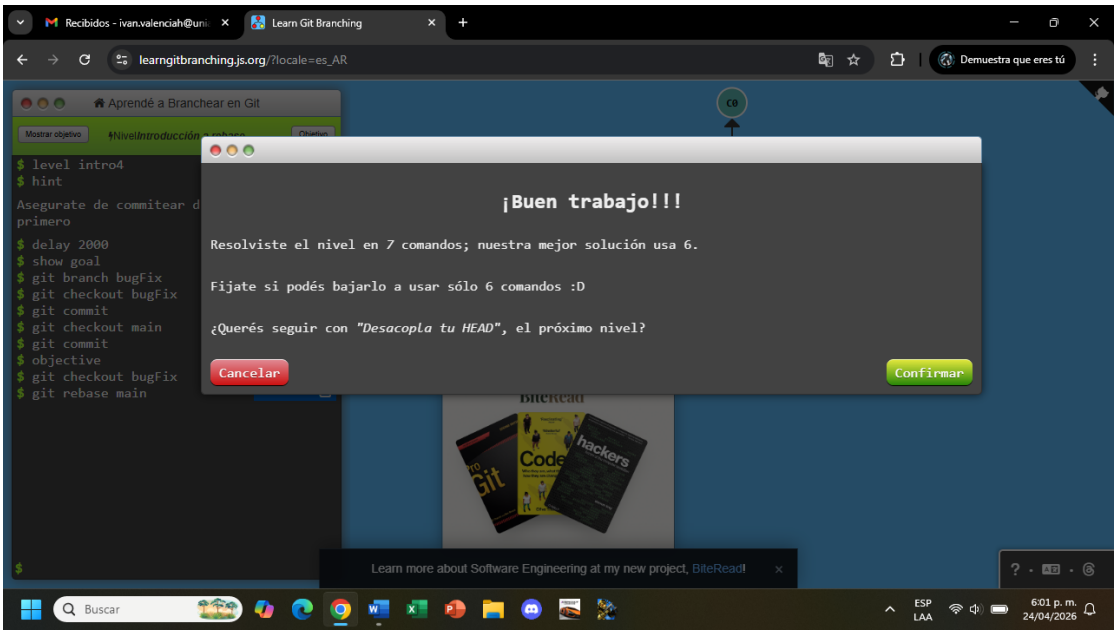
git checkout “nombre”: Cambia entre ramas o versiones del proyecto.

El tercer nivel es “Mergeando en Git”.



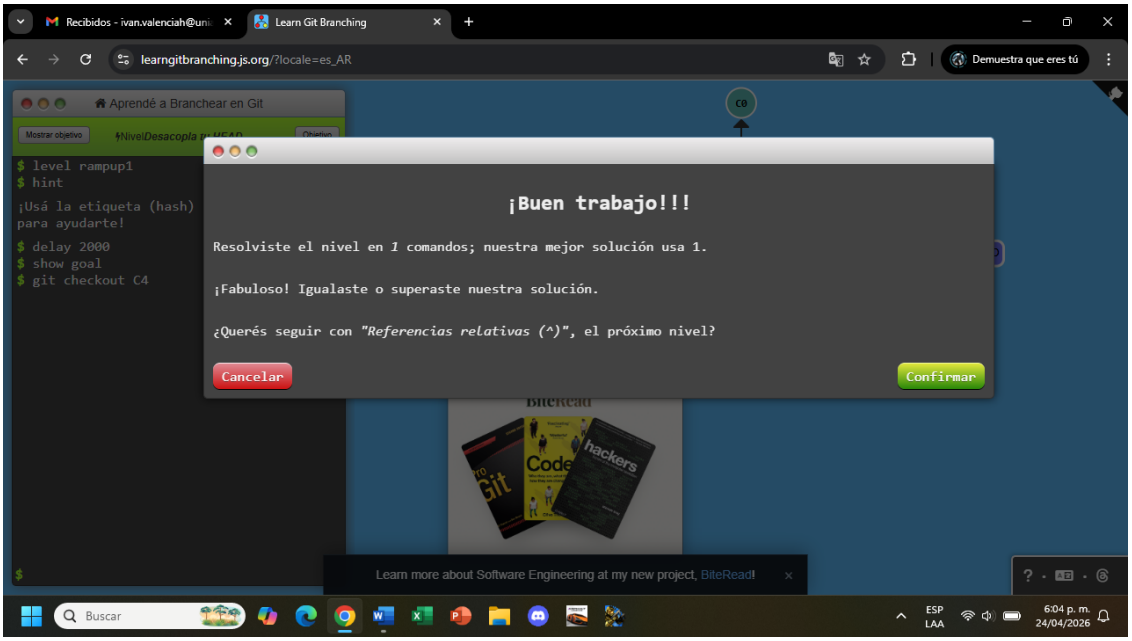
git merge “nombre”: Une una rama con la rama actual o principal “main”.

El cuarto nivel es “Introducción a rebase”.



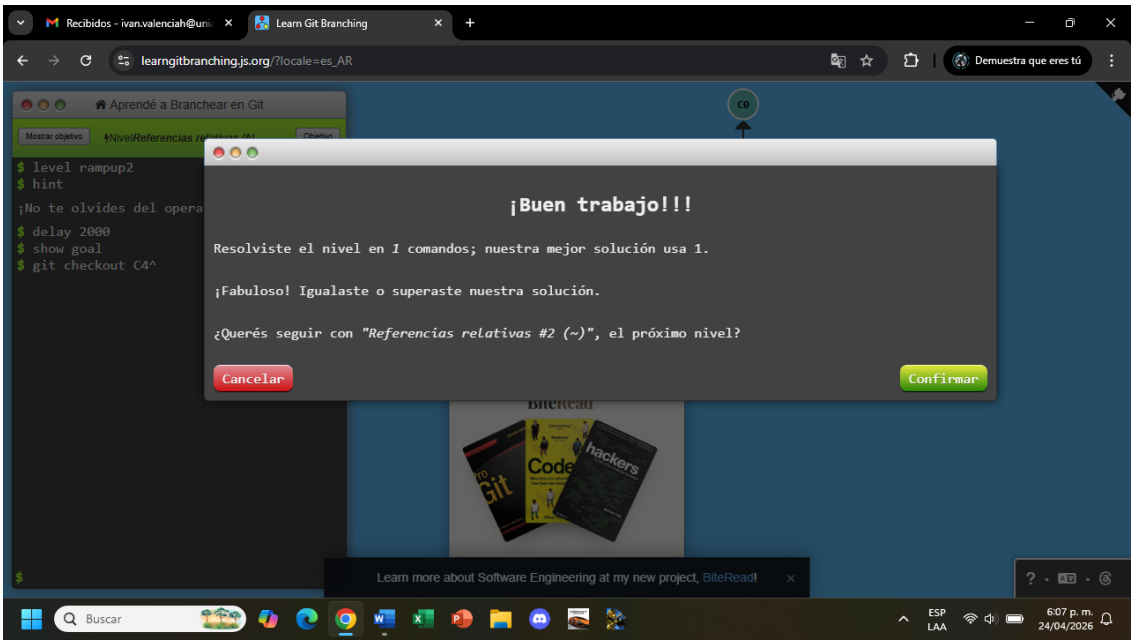
git rebase “nombre”: Mueve o reaplica los commits encima de otra rama para mantener un historial más limpio.

Continuamos con el segundo capítulo llamado “Acelerado” y el primer nivel es “Desacopla un HEAD”.



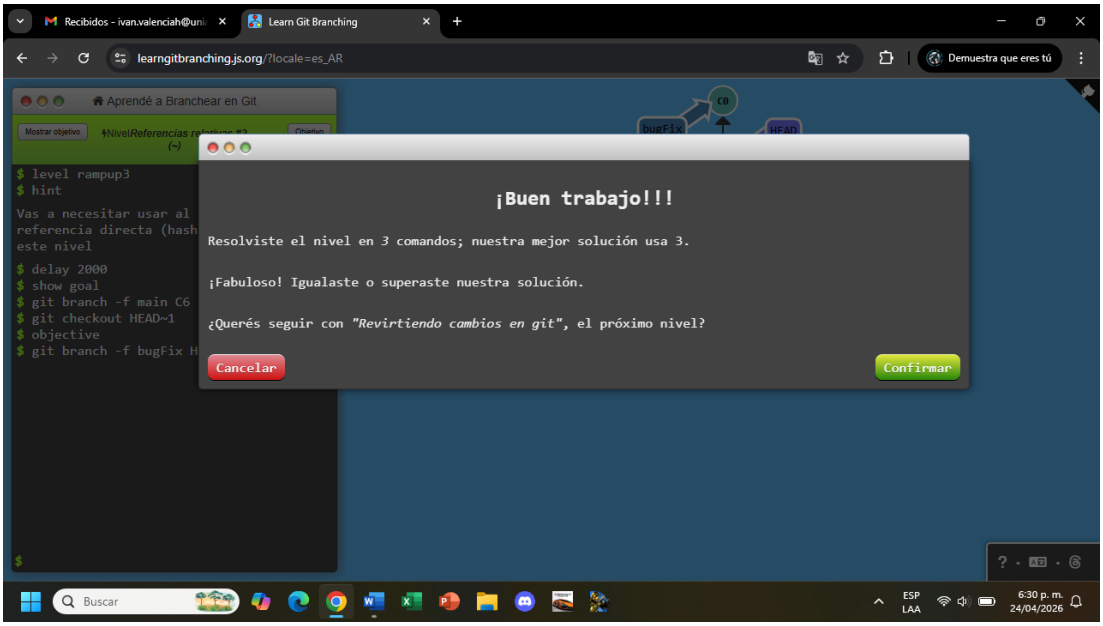
HEAD es un nombre como representación que tiene el commit que se tiene seleccionado actualmente.

El segundo nivel es “Referencias relativas (^)”.



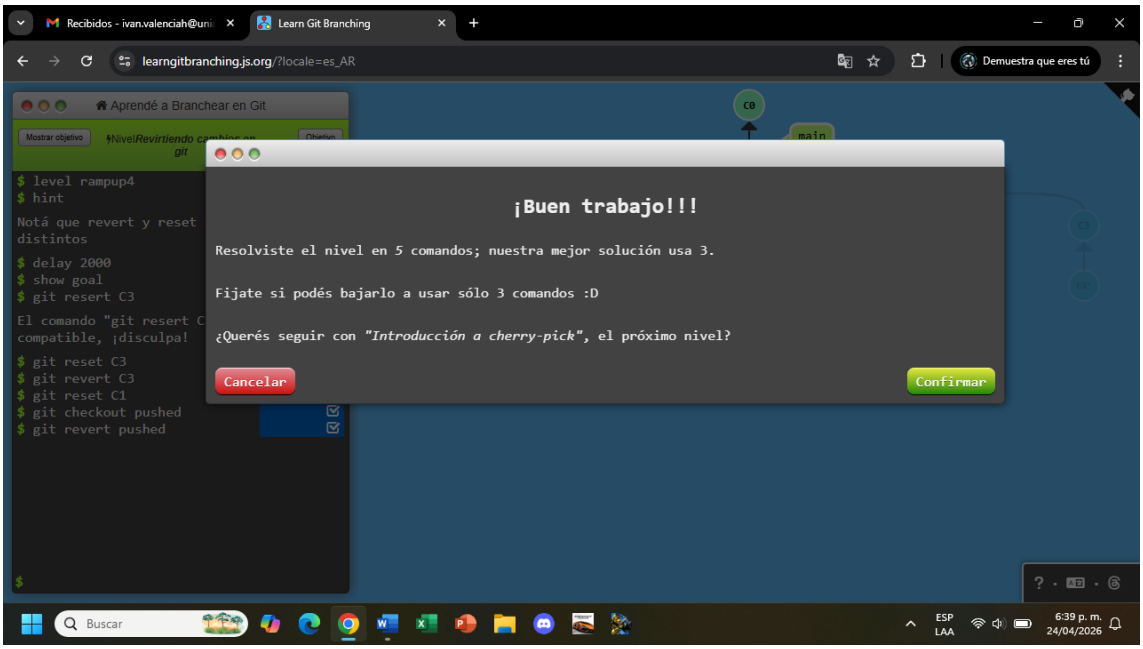
En la explicación se menciona que trabajar con los hash no es tan lindo ni bonito en la terminal y son hashes bastante largos, pero con cierto rango de caracteres se puede identificar el commit directamente con (^) o (~<num>).

El tercer nivel es “Referencias relativas #2 (~)”.



git branch -f bugFix HEAD-1: Con este comando se puede mover la rama forzadamente 1 o 3 commits arriba sobre el HEAD.

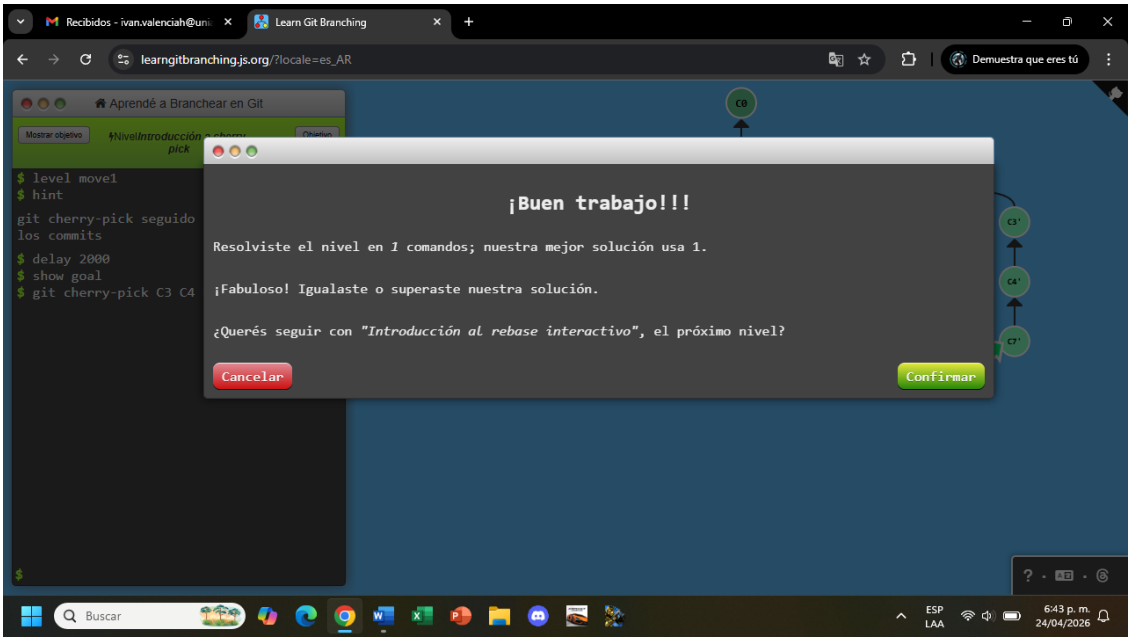
El cuarto nivel es “Revirtiendo cambios en git”.



git reset: Elimina el ultimo commit y regresa el proyecto al estado anterior.

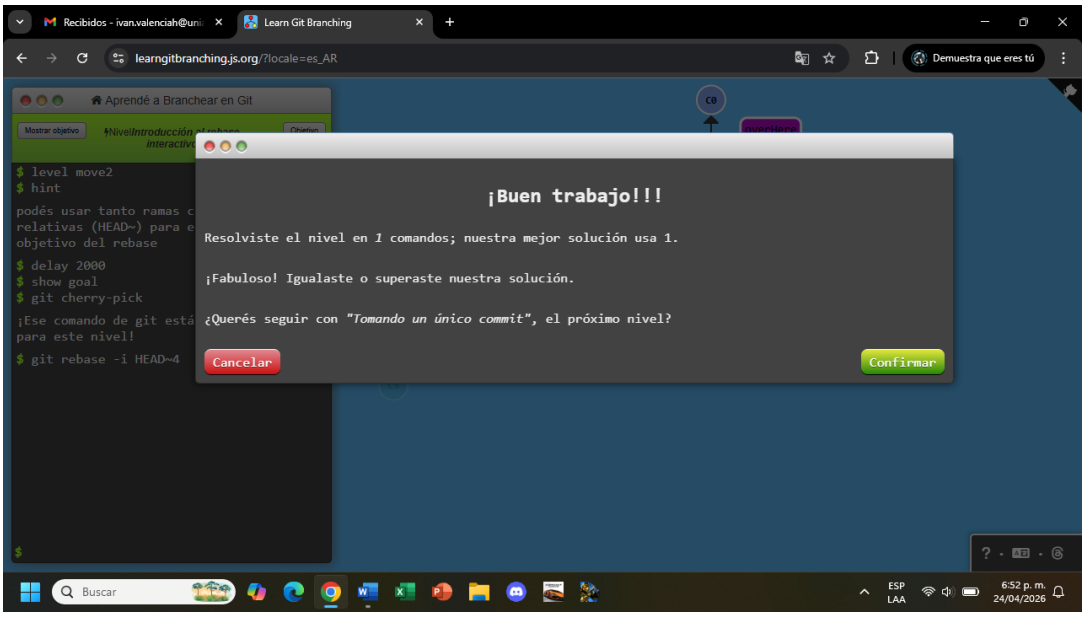
git revert: Crea una copia de un commit de la reversión que se hizo.

El tercer capítulo se llama “Moviendo el trabajo por ahí” donde el primer nivel es “Introducción a cherry-pick”.



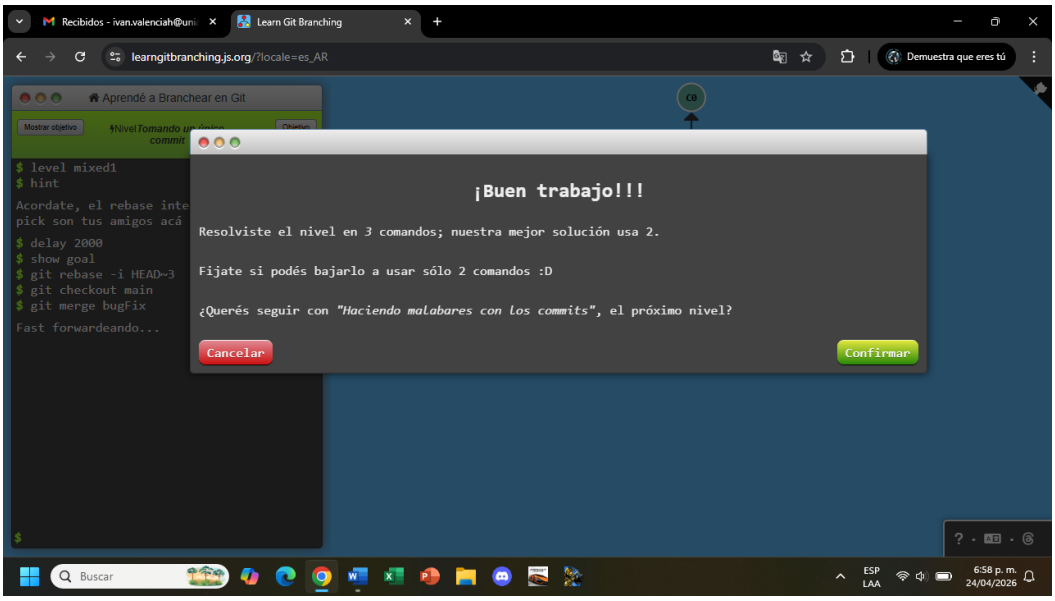
git cherry-pick: Copia un commit de otra rama y lo aplica en la rama “main”.

El segundo nivel es “Introducción al rebase interactivo”.



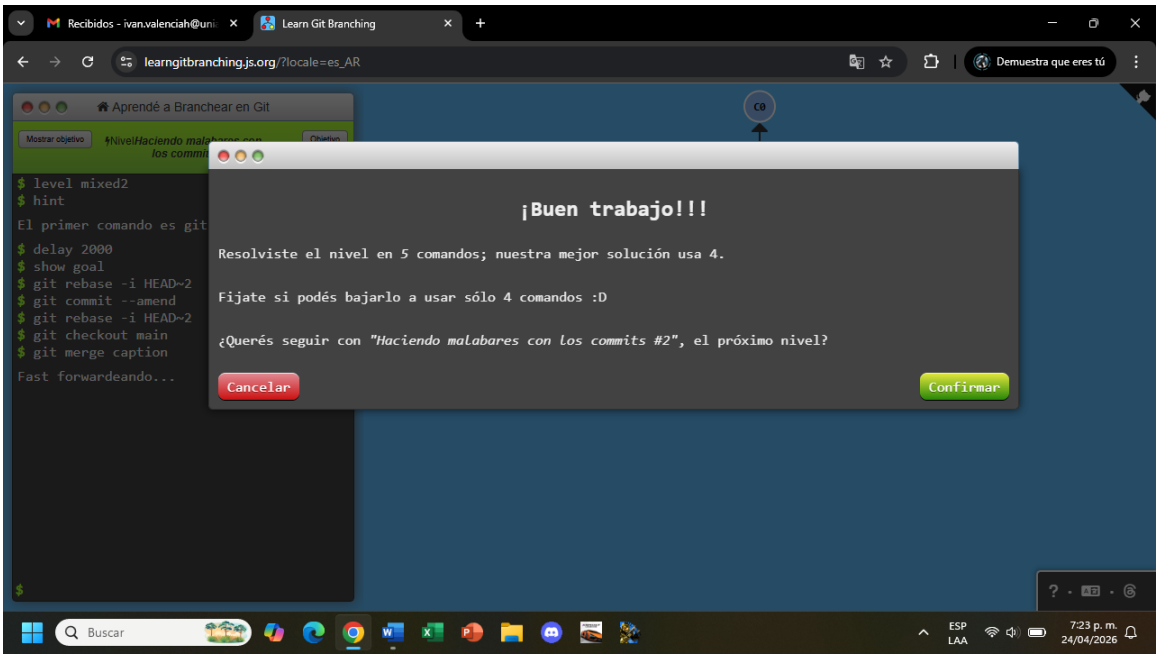
Este comando sirve a ayudarnos cuando no sabemos que commits quiero. El comando es usando el comando rebase pero con -i. En este caso se puede hacer con una UI interactiva por medio de un drag and drop.
git rebase -i HEAD-4: Se pueden acomodar los commits de manera más fácil.

El cuarto capitulo es “Bolsa de gatos” y el primer nivel se llama “Tomando un único commit”.



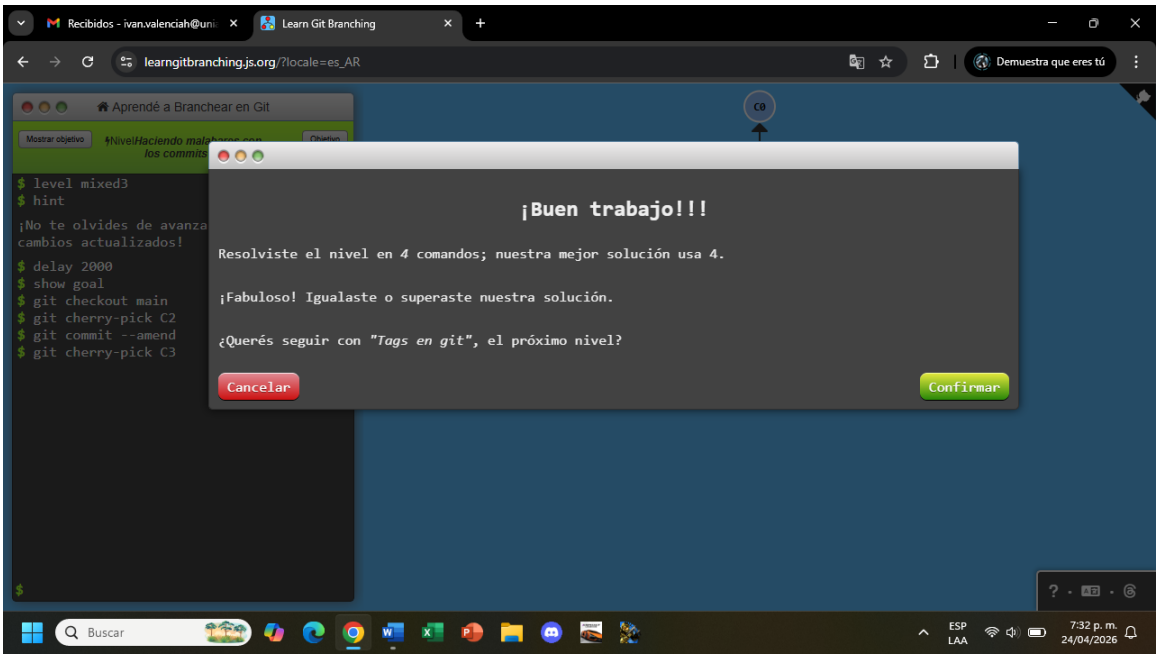
En este desafío se podía utilizar comandos anteriores para mover commits entre ramas.

El segundo nivel es “Haciendo malabares con los commits”.



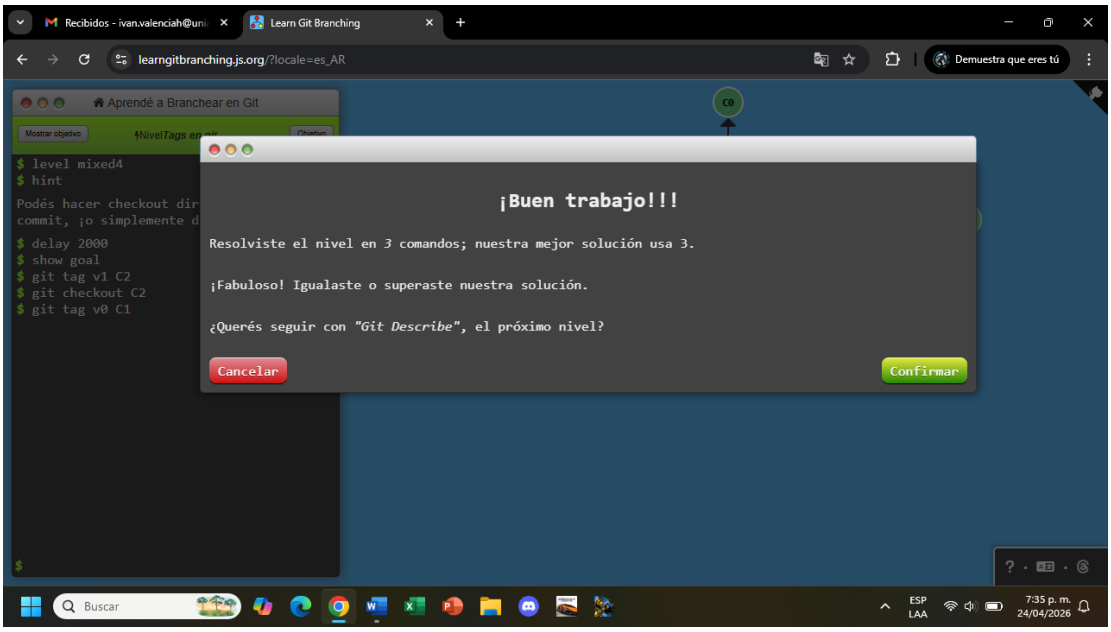
En este desafio es moverse entre ramas con git rebase -i, después con git commit –amend se aplica la ligera modificación, luego reordenamos los commits con git rebase -i y por último movemos a main a la parte actualizada del árbol de commits.

El tercer nivel es “Haciendo malabares con los commits #2”.



En este desafío es moverse entre ramas con git cherry-pick.

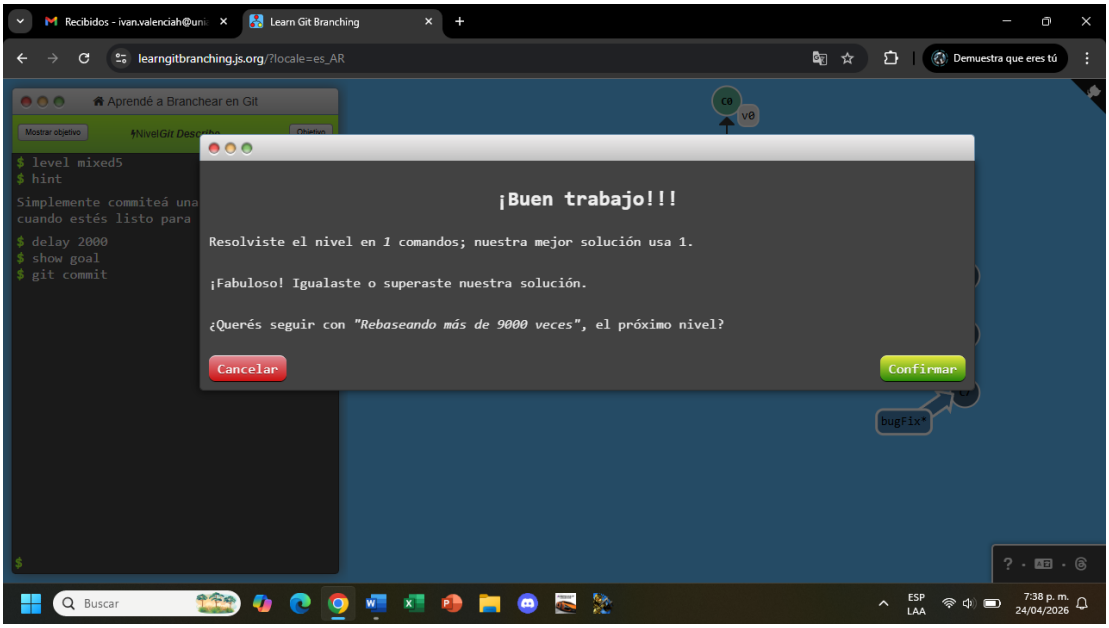
El cuarto nivel es “Tags en git”.

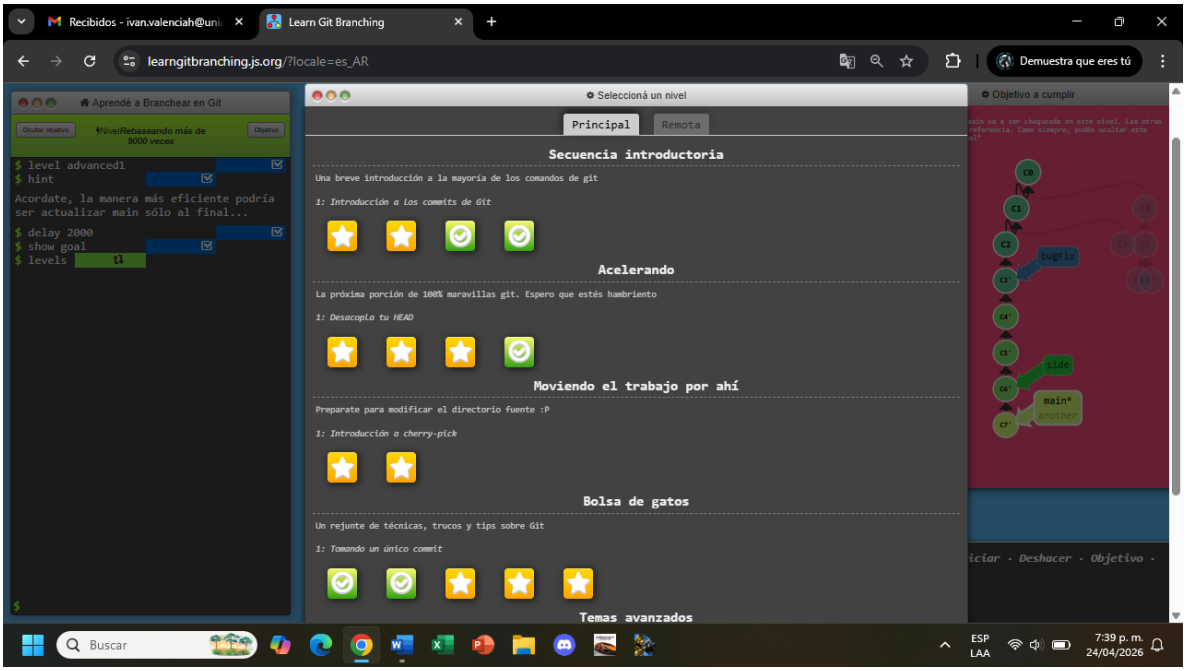


Los tags en los commits sirven para marcarlos permanentemente como etiquetas.

git tag v1 C2: Es para nombrar una etiqueta (v1) a un commit (C2) y marcar una versión específica en el proyecto.

El quinto nivel es “Git Describe”.



git describe: Ayuda a describir dónde está la etiqueta (tag) más cercano después de que uno se mueve en commits ya sea hacia delante o hacia atrás.

Aquí se muestra los ejercicios de Comando de Git solicitados ya están completados.